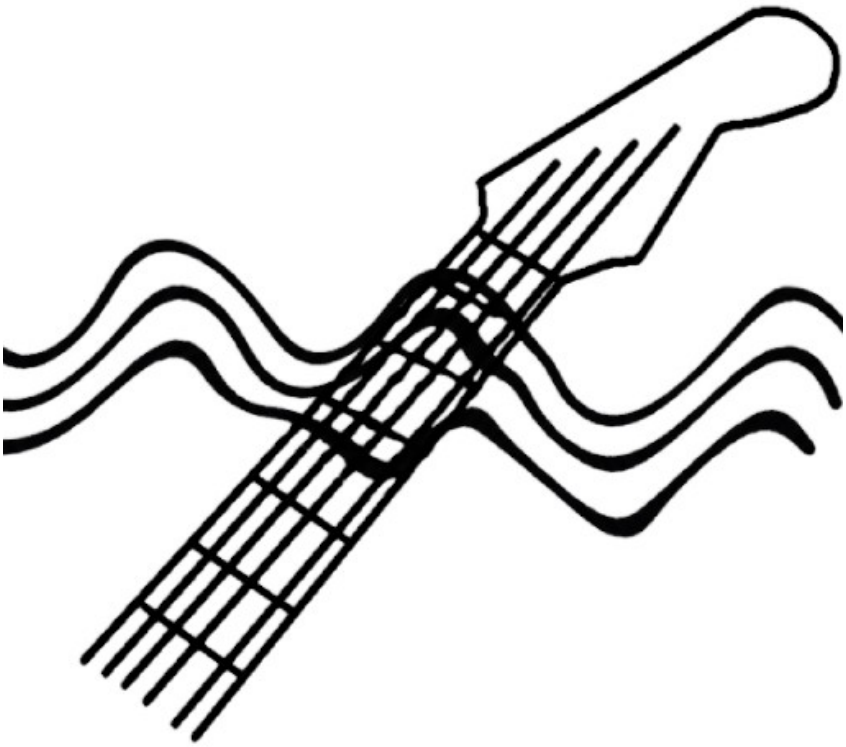
Project Specification

Software Development Year 4 – Project



Author: Mark Lambert
Student ID: C00192497

Start Date: 9th September 2026

Project Title

Guitar Chord & Skill Analysis Tool (*Chroma **OR** Fretwave **OR** Signical*)

Project Summary

A mobile application that performs analysis on guitar audio to identify chords being played, measure the timing of transitions between chord changes, and use that data to estimate a player's skill level. Based on the users skill level, the system recommends scales, exercises or songs which are within the player's skill level.

Functionality

- Detect chords played from audio input and use audio processing tools
- Measure the time taken between chord transitions
- Assign a skill estimate based on chord difficulty and transition times (barre chords much more difficult than open cowboy chords)
- Recommend exercises, scales or songs matched to the estimated skill level
- The application should run on a smartphone

Features

- **Audio Input –** User provides a recording/records with phone mic live
- **Chord Detection –** the system identifies the chords being played throughout recording
- **Analysis –** the system measures the time between chord changes
- **Skill Assigning –** based on chord difficulty and transition speed, a skill score is calculated
- **Recommendations –** a dataset of scales, exercises and songs are assigned to the user based on their skill score (Beginner | Intermediate | Advanced)
- **Mobile UI –** mobile based interface for recording and uploading audio
- **Progress Tracking –** user can track previous practice history and recommendations
- **Personal Feedback –** Using Ollama's local models, the results from the users audio can be passed to a template prompt which generates personalised reports (Backup generalised messages needed and some form of filtering to ensure the AI hasn't strayed off topic)

Tech Stack

Audio processing – Python, librosa, noisereduce | librosa has chroma feature and note onset detection, noisereduce for audio sanitisation
Backend - Flask
Frontend – Flutter (cross platform in one codebase) | Handle the audio input and displays results
Database – MariaDB | Store user practice history and skill scores
IDE – PyCharm, VsCode (Flutter)
Version Control – GitHub
Data – Kaggle/GuitarSet [1]
LLM Integration – Ollama | Could query Ollama's local REST API on small models like llama3/phi3

The chord detection approach implemented in my prototype follows the standard chroma matching baseline highlighted by Snyder [2], by comparing a frame's chroma vector against a bank of major/minor templates.

Why?

Existing guitar learning applications such as Yousician and Fender Play do provide feedback, however, the user must follow their own predefined curriculum via video/lesson. The application then checks their playing against a set of targets. This forces the user into a restricted set of rules/exercises they must follow.

This application will generate feedback from the guitarists own unscripted playing. A user can play a single chord change of their choosing, and the system will analyse and provide feedback based on the transitions. This freedom will in-turn provide a more realistic insight on the guitarists ability. Other apps force the guitarist into following a lesson slightly above their skill level, resulting in them "overplaying" and making mistakes due to tension in the body. The objective of this application is to encourage **good form** over **speed**. Sloppy chords, with bad notes can be detected via the output of the chroma vectors.

Feedback is generated from the following

- **Transition Speed –** Relative to the difficulty of chord (open-to-open chord? Open-to-barre? Barre-to-barre? Diminished/augmented? Relative to current hand position?)
- **Consistency –** If more than one chord change, the users average transition time will have an affect on their score

These parameters will be queried against the expected transition speed benchmarks for practical, bad-habit free improvement from established beginner guitar teacher JustinGuitar [3]

He states that a guitarist should choose two chords that they find tricky to switch between, and play for 60 seconds, see how quickly they can transition while focusing on **quality over speed.** By repeating the exercise and logging the data, the results will build a profile on the users progress.

Sandercoe states that the long term goal is "60 changes in a minute". And that to pass a first grade level guitar exam "thirty changes in a minute" would be expected. [4]

Using these numbers, we can assume an Advanced guitarist can perform a chord transition in one second, depending on difficulty. A beginner guitarist would be expected to transition from easier chords in roughly two seconds.

What's Required From the User

The user is required to select one of three options as their own interpretation of their skill level, Beginner | Intermediate | Advanced. This will be used as a baseline for their expected skill results.
The user is required to have a mobile device with Android or iOS.
The user is required to have an acoustic OR electric guitar and a functioning mobile microphone.

Targets (MoSCoW)

- **MUST –** Chord detection and timing measurement works on a small set of defined open chords, basic scoring, working on mobile
- **SHOULD –** Larger set of chords (barre), ability to save data and track progression
- **COULD –** ML based detecion to remove templated skill assignment
- **WOULD –** Ollama integration for personalised feedback

References

[1] Vinci, F (2021) *Guitar Chords V4* [Kaggle] Available at: https://www.kaggle.com/datasets/fabianavinci/guitar-chords-v3 (Accessed: 12th September)

[2] Snyder, W (2021) *"CS 583 – Computational Audio: Lecture 10 – Chroma, Chord Recognition by Template Matching, Problems with Chord Recognition, Coding the Algorithm"* [Boston University] Available At: https://www.cs.bu.edu/fac/snyder/cs583/Lectures/Lecture 18.pdf (Accessed: 12th September 2026)

[3] Sandercoe, J (2020) "*Perfect Fast Changes"* Available at: https://www.justinguitar.com/guitar-lessons/perfect-fast-changes-b2-802 (Accessed: 11th September 2026)

[4] Sandercoe, J (2020) *"Perfect Fast Changes"* [Video] Youtube. Available at: https://www.youtube.com/watch?v=xSFHgeJUuIs (Accessed: 11th September 2026)

[5] Lynn Zheng (2021) *"How to use Python to Detect Music Onsets"* [freeCodeCamp] https://www.freecodecamp.org/news/use-python-to-detect-music-onsets/#:~:text=Finally,above%20for%20Aubio (Accessed: 06th October 2026)

[6] tombatossals (2025) "*chords-db"* [github] Available at: https://github.com/tombatossals/chords-db (Accessed: 06th October 2026)